



TA-LIB各類重要技術指標

TA-Lib 內建大約 150+ 種技術指標，實務上常用的可以分成幾大族群來抓重點。

1. 趨勢類 (Trend / Overlap)

主要用來判斷多空趨勢與支撐壓力。

- MA / SMA / EMA / WMA / DEMA / TEMA：各種移動平均線，用於判斷多空方向與支撐壓力。^[1]
- BBANDS：布林通道，標準差區間，常用來看壓力、支撐與波動度。^[1]
- SAR / SAREXT：拋物線轉向指標，常用來做移動停損與多空翻轉點。^[1]
- MIDPOINT / MIDPRICE：一段期間的中位價或中間價，用來平滑價格或當作參考軸。^[1]
- HT_TRENDLINE、MAMA、T3：比較進階的趨勢線與自適應均線，用於平滑趨勢、減少假訊號。^[1]

這一類通常會搭配價格一起畫在同一張 K 線圖上 (Overlap Studies Functions)。

2. 動能/震盪類 (Momentum / Oscillators)

主要用來判斷超買超賣、強弱與反轉。

- RSI：相對強弱指標，0-100，超買超賣判斷最常見的指標之一。^[2]
- MACD / MACDEXT / MACDFIX：快慢 EMA 差與訊號線，用來看趨勢強度與多空交叉。^[1]
- STOCH / STOCHF / STOCHRSI：KD 指標與隨機 RSI，反應速度比 RSI 更快。
- CCI：順勢指標，偏離平均價的程度，用來抓過熱與反轉。^[1]
- MFI：量價結合的 RSI（資金流量指標），常用於判斷量價是否配合。^[1]
- AROON / AROONOSC：阿隆指標，用來判斷是否處於趨勢或盤整。^[1]
- ADX / ADXR / DX：平均趨向指數系列，只看趨勢強度、不看方向，適合用來判斷「是否適合用趨勢策略」。^[1]
- APO / PPO：絕對／百分比價格振盪器，類似 MACD 的概念。^[1]
- CMO、ROC、MOM、WILLR 等：各種動能與擺動指標，用來看價格變化速度與超買超賣。^{[3] [1]}

這一類大多畫在價格下方獨立區域，用於找到進出場區間或過熱區。

3. 成交量/資金類 (Volume)

用成交量與價格關係判斷買賣力道。

- OBV：能量潮，累計成交量方向，用於判斷量是否支撐趨勢。^[3]
- AD / ADOSC：累積／派發線，量價結合，偏向資金流入流出判斷。
- CMF (如用 pandas_ta) 、 MFI：量價版 RSI。TA-Lib 裡 MFI 直接可用；CMF 多用其他庫或自寫。^{[4] [1]}

這一類適合搭配價量背離去做判斷。

4. 波動度／風險類 (Volatility)

用來衡量市場波動與風險大小。

- ATR：平均真實波幅，常用來設停損距離與部位 sizing。^[3]
- NATR：標準化 ATR，便於不同標的比較。
- TRANGE：真實波幅本身，可以再加工成其他指標。
- BBANDS 同時也可視為波動度指標（標準差通道）。^[1]

量化策略中，ATR 幾乎是風控與動態槓桿配置的標配。

5. 價格轉換 / 價格型指標 (Price Transform)

這些本身不一定是「指標」，而是方便做後續計算的轉換。

- AVGPRICE、TYPPRICE、WCLPRICE、MEDPRICE：各種加權價格（如典型價、收盤與高低價平均）。
- HT_DCPERIOD / HT_DCPHASE / PHASOR 等：希爾伯特變換相關，用於更進階的週期分析。^[1]

通常在自訂因子或自寫指標時蠻實用。

6. 統計類 (Math / Statistic Functions)

用來做標準差、相關性等統計性技術指標。

- STDDEV：標準差，可自訂期間與權重，常用於波動度、風險或布林帶類改寫。^[1]
- VAR、CORREL：變異數與相關係數，用於雙標的關係分析或 Pair Trading。
- LINEARREG、TSF：線性回歸線與時間序列預測線，可當作動態趨勢線。

這些對你做多因子或量化回測特別有用。

7. 如何在策略裡抓「重要那幾個」

以實務經驗，多數交易者真的長期會用且有效率維護的，大概不會超過 5-8 個指標，你可以先挑下面這組作為「核心 TA-Lib 指標集」：

- 趨勢：EMA / SMA、BBANDS、SAR。^[1]
- 動能：RSI、MACD、STOCH (或 STOCHRSI) 、CCI。^{[2] [1]}

- 波動 / 風控：ATR。 [3]
- 量能：OBV、MFI。 [3] [1]

其他像 ADX、AROON、STDDEV、LINEARREG 可以當作進階加分因子，用在選股或風險調整上。

如果你願意，我可以下一步幫你設計一個「TA-Lib 指標模組介面」，把這些常用指標包成統一的 Python 函數（含多標的、多周期、輸出成 DataFrame），方便直接接到你的回測或 live 系統。

✻

更詳細

下面整理成「分類 → 指標列表 → 功能 + 常見用法」，偏向你實務量化會用得到的細節（中文名、適用場景、常見參數習慣）。 [10] [11] [12]

1. 重疊指標 (Overlap Studies)

這類是「畫在 K 線上」的線與通道，用於趨勢、支撐壓力、平滑價格。 [12] [10]

- MA / SMA / EMA / WMA / DEMA / TEMA / TRIMA / KAMA / T3 / WMA
 - 功能：各種移動平均線，差別在平滑程度與對新資料的敏感度。 [10]
 - 實務：
 - 趨勢判斷：短均線 > 長均線 多頭，反之空頭（如 5/20、20/60）。
 - 均線系統：穿越 + 乖離過大（如 close 與 20 日 MA 的百分比）。
- BBANDS (布林通道)
 - 功能：中軌為 MA，上下軌為 $MA \pm N$ 倍標準差，反映波動度。 [10]
 - 實務：
 - 盤整：價格在上下軌之間震盪，上下軌收斂。
 - 突破：價格有效突破上軌或下軌 + 放量，作為趨勢起漲 / 起跌信號。
- SAR / SAREXT (拋物線轉向)
 - 功能：以點狀跟隨價格的動態追蹤停損線。 [10]
 - 實務：
 - 設移動停損：long 時價格跌破 SAR 出場，short 時價格突破 SAR 出場。
- MIDPOINT / MIDPRICE / AVGPRICE / MEDPRICE / TYPPRICE / WCLPRICE
 - 功能：各種「平均價格」轉換（高低平均、HLC 平均等）。 [12] [10]
 - 實務：
 - 作為自定指標輸入（例如用 TYPPRICE 取代 close），或做中軸線。
- HT_TRENDLINE (希爾伯特趨勢線)
 - 功能：利用希爾伯特變換產生接近「理想平滑」的趨勢線。 [10]
 - 實務：

- 對抗噪音的趨勢線，用來比對價差或當 filter。

2. 動量／震盪指標 (Momentum Indicators)

這類多是「0-100 或中性線」的震盪器，用於強弱與超買超賣、反轉訊號。^{[11] [12] [10]}

- RSI (Relative Strength Index)
 - 功能：0-100，衡量上漲與下跌的平均幅度比重。^[10]
 - 常見用法：
 - 超買超賣：>70 超買，<30 超賣（強趨勢可放寬到 80/20）。
 - 背離：價格創新高，RSI 沒創新高 → 多頭背離。
- STOCH / STOCHF / STOCHRSI (KD 與隨機 RSI)
 - 功能：將收盤價位置相對於一定期間高低區間標準化到 0-100。^[10]
 - 實務：
 - KD 黃金交叉／死亡交叉 + 區間 (<20 或 >80) 判斷進出場。
 - STOCHRSI 用在短線，對價格變化更敏感。
- MACD / MACDEXT / MACDFIX
 - 功能：長短期 EMA 差值 (DIF)、其平均 (DEA/Signal)，以及柱狀圖 (Hist)。^[10]
 - 實務：
 - 0 軸上多頭，0 軸下空頭。
 - DIF 與 DEA 交叉、柱狀圖由負轉正 (多)、由正轉負 (空)。
- CCI (Commodity Channel Index)
 - 功能：衡量價格偏離其平均 (通常是典型價)，常在 ±100 區間。^[10]
 - 實務：
 -  100 視為多頭強勢區，<-100 視為空頭強勢區。
 - 極端值回落常作為反轉訊號。
- ADX / ADXR / DX / PLUS_DI / MINUS_DI
 - 功能：Directional Movement 系列，處理由 Wilder 提出，用來測趨勢「強度」而非方向。^[10]
 - 實務：
 - ADX 高 (如 >25) 代表有趨勢，低 (<20) 代表盤整。
 - PLUS_DI 與 MINUS_DI 的交叉可看多空主導。
- AROON / AROONOSC
 - 功能：觀察一段期間內最新高／低點出現的位置距離現在有多近。^[10]
 - 實務：
 - Aroon Up 接近 100、Aroon Down 接近 0，趨勢向上，反之亦然。

- MOM / ROC / ROCP / ROCR / ROCRI100
 - 功能：單純的動能與報酬率計算（價差、百分比變化）。^[10]
 - 實務：
 - 自定因子（如近 N 日動能），或做 breakout 的 filter。
- CMO (Chande Momentum Oscillator) 、WILLR、ULTOSC、TRIX ...
 - 功能：不同公式的動能／震盪器，用法與 RSI/KD 類似。^{[11] [10]}
 - 實務：
 - 你可以只挑一兩個熟悉的（如 RSI + CCI），避免指標重複。

3. 成交量指標 (Volume Indicators)

這類把「量」和「價」結合，判斷資金流向與趨勢可信度。^{[12] [10]}

- OBV (On Balance Volume)
 - 功能：若收盤價上漲就加上當日量，下跌就減去當日量，累積成一條線。^[10]
 - 實務：
 - 價漲量增、OBV 同步創高，趨勢可信。
 - 價漲量縮或 OBV 未創新高，警惕背離。
- AD (Chaikin A/D Line) / ADOSC (Chaikin A/D Oscillator)
 - 功能：將量按照收盤價在高低區間中的位置加權後累計，反映「買盤／賣盤壓力」。^[10]
 - 實務：
 - AD 線向上代表資金「吸籌」，向下代表「出貨」。
- MFI (Money Flow Index)
 - 功能：量價版 RSI，使用典型價 $\times$ 量計算正負資金流。^{[12] [10]}
 - 實務：
 - 超買超賣區邏輯與 RSI 類似，只是更注重量。
- BOP (Balance of Power)
 - 功能：用開高低收計算多空力量平衡。^[10]
 - 實務：
 - 多用來做精細的 K 線強弱判斷，搭配量能。

4. 波動度指標 (Volatility Indicators)

主要用來量化「當前市場波動程度」，對停損、槓桿與部位 sizing 很關鍵。^{[11] [12] [10]}

- ATR / NATR / TRANGE
 - TRANGE：當日真實波幅 ($\max(\text{high-low}, \text{abs}(\text{high-prev_close}), \text{abs}(\text{low-prev_close}))$)。^[10]

- ATR：TRANGE的移動平均，平滑的波動度。^[10]
- NATR：標準化 ATR ($ATR / close \times 100$)，便於不同標的比較。^[10]
- 實務：
 - 停損：以 1-3 倍 ATR 定義技術性停損距離。
 - 部位：波動高時縮小部位，波動低時放大部位。
- BBANDS (另屬 Overlap，但常當波動指標使用)
 - 上下軌擴張代表波動放大，收斂代表波動降低。^[10]

5. 價格轉換 (Price Transform) 與週期指標 (Cycle Indicators)

多用於進階指標設計、週期分析。^{[12] [10]}

- AVGPRICE、TYPPRICE、MEDPRICE、WCLPRICE
 - 功能：不同組合的加權價格 (如 $TYPPRICE = (high + low + close) / 3$)。^[10]
 - 實務：
 - 替代 close 當作輸入，會讓指標較不受單一價位噪音干擾。
- HT_DCPERIOD / HT_DCPHASE / HT_PHASOR / HT_SINE / HT_TRENDMODE
 - 功能：Hilbert Transform 系列，用來估計主導週期、相位與趨勢／循環模式。^{[11] [10]}
 - 實務：
 - 週期策略、循環股、季節性研究的進階工具。

6. 統計／數學函數 (Statistic & Math Functions)

給你在 TA-Lib 裡直接做基本統計計算，常用來構造 factor。^{[11] [12] [10]}

- STDDEV / VAR
 - 功能：期間標準差與變異數。^[10]
 - 實務：
 - 自製波動度指標、風險度量 (如 rolling std of returns)。
- CORREL (皮爾森相關係數)
 - 功能：兩組序列的滾動相關係數。^[10]
 - 實務：
 - Pair Trading、類股輪動相關性分析。
- BETA
 - 功能：標的報酬相對基準 (如指數) 的 beta。^[10]
 - 實務：
 - 風險調整、組合建構時規範總體 beta。
- LINEARREG / LINEARREG_SLOPE / LINEARREG_ANGLE / LINEARREG_INTERCEPT / TSF

- 功能：線性回歸擬合價格路徑，給出趨勢線、斜率、角度、時間序列預測線。^[10]
- 實務：
 - 把斜率當作「趨勢強度 factor」，或用 TSF 作為平滑趨勢線。
- SUM / MAX / MIN / MINMAX / MININDEX / MAXINDEX
 - 功能：期間加總、最大值、最小值與其索引。^[10]
 - 實務：
 - 很適合在 TA-Lib 內完成資料前處理，例如累計量、期間最高價等。

7. K 線形態 (Pattern Recognition)

這一類在 TA-Lib 特別多 (CDL 開頭)，全部都是「K 線形態偵測」，回傳整數分數 (多為 -100, 0, 100)。^{[12] [10]}

- 代表性指標：
 - CDLDOJI、CDLHAMMER、CDLSHOOTINGSTAR、CDLENGULFING、CDLMORNINGSTAR、CDLEVENINGSTAR、CDL3WHITESOLDIERS、CDL3BLACKCROWS 等。^[10]
- 實務：
 - 多用於尋找反轉或延續的 K 線形態，可作為事件驅動 signal 或與其他因子混用。
 - 因為雜訊高，通常會加 filter (如只在 ADX 低時看反轉形態)。

8. 常見參數與組合建議

給你一組較常見、容易移植到系統裡的「標準參數集」，再自行微調：

- 均線：
 - 短：5、10 日 (台股日線)、20 日 (均線趨勢)。
 - 中：60 日 (季線)、120 日。
 - 長：240 日 (年線)。
- RSI：14 期，水位 30/70 或 20/80。^[10]
- KD (STOCH)：fastk_period=9, slowk=3, slowd=3, 區間 20/80。
- MACD：快線 12、慢線 26、Signal 9 (TA-Lib 預設)。^[10]
- ATR：14 期，用作停損距離或波動調整。
- BBANDS：20 日 MA，標準差 2 倍。

常用組合示例 (供你接回測框架)：

- 趨勢策略：EMA(20, 60) + ADX + ATR (趨勢方向 + 有無趨勢 + 波動控槓桿)。
- 區間反轉：RSI(14) + STOCHRSI(14) + BBANDS (超買超賣 + 區間邊界)。
- 量價驗證：OBV + MFI + 價量背離檢測。

如果你願意，我可以直接幫你寫一個 `build_talib_features(df, config)` 的範例框架（支援多標的、多時間窗、輸出 MultiIndex columns），把上面這些指標全部模組化成「配置檔驅動」，你只要給 symbol 列表與欄位命名規則就能接進你的回測 / 交易系統。



連動性的圖有哪些 誰領先誰是否有是領先指標判讀 還是同時 還是落後 如何做

在圖形與統計上談「連動性」，可以分三個層次：概念（領先/同時/落後）、視覺圖型怎麼畫、以及實際量化怎麼判斷誰領先誰。下面用你熟悉的量化語言整理。 [25] [26]

1. 領先 / 同時 / 落後 的嚴格定義

- 領先指標 (Leading)
某變數 X 的變化，系統性地出現在 Y 變化「之前」一段固定時間，例如 X 提前 1-3 期轉折， Y 才跟著轉。 [27] [25]
- 同時指標 (Coincident)
 X 與 Y 的高低點、轉折點大致在同一時間發生，沒有明顯穩定的時間差。 [25] [27]
- 落後指標 (Lagging)
 X 系統性地在 Y 變化「之後」才變化，例如景氣已轉差一陣子，失業率才明顯上升。 [28] [25]

重點：這三者沒有絕對標準答案，要用「長期資料 + 統計方法」來驗證，而不是用肉眼看幾張圖就決定。 [29] [25]

2. 連動性的常見圖形

你問的「連動性的圖有哪些」，在實務上主要有：

1. 疊圖 (Overlay chart)

- 同一張圖上畫兩個價格（或指標）曲線，有時會加第二 Y 軸。
- 用途：直觀看趨勢方向是否一致、有沒有明顯前後時間差。

2. 走勢 + 位移疊圖 (Time-shift overlay)

- 人為把其中一條線往左/往右平移幾期，再疊在一起，觀察是否「對齊」得更好。
- 用途：肉眼初判可能的 lead/lag 長度，例如台指與電子指數。

3. 散佈圖 (X-Y scatter)

- X 軸放一個變數（或其變化率）， Y 軸放另一個，通常加上迴歸線。
- 用途：看「相關性強不強」，但此圖本身不含時間先後資訊。

4. 交叉相關圖 (Cross-correlation function, CCF)

- X 軸是時間差 k （負 lag = X 領先，正 lag = Y 領先）， Y 軸是每個 lag 下的相關係數 $\rho(k)$ 。 [26] [30]

- 用途：這是最直接看「誰領先誰」的量化圖，峰值出現在哪個 lag，就是最可能的領先/落後關係。

若你想在 TradingView 這種平台上做視覺化，可以直接用現成的 cross-correlation 指標（例如有人寫的 Cross-Correlation Lead/Lag Analyzer），它就是畫 rolling cross-correlation 在多個 offset 上，幫你顯示哪個 lag 的相關度最高。^{[30] [31]}

3. 如何量化判斷「誰領先誰」

核心工具就是「滯後 / 領先項」與「交叉相關」。^{[29] [26]}

1. 建立不同 lag 的序列

- 給兩個時間序列 X_t, Y_t （可以是價格、報酬、指標值）。
- 建立 X_{t-k} 或 Y_{t-k} ($k > 0$ 代表 X 領先 k 期, $k < 0$ 代表 X 落後)。^[29]

2. 計算各 lag 的皮爾森相關係數

- 對每個整數 lag $k \in [-K, K]$ 計算 $\rho(k) = \text{corr}(X_{t-k}, Y_t)$ (或相反)。^[26]
- 把 $\rho(k)$ 畫成一條線，就是 cross-correlation function。

3. 判讀規則（實務）

- 若在某個正 lag $k^* > 0$ 有明顯最高相關（且顯著）：
 - 代表「X 的變化通常在 k^* 期之前發生，Y 才跟」→ X 領先 Y 約 k^* 期。
- 若高峰在負 lag，則 Y 領先 X。
- 若峰值在 0 附近，且左右對稱，較像是「同時指標」。^[26]

4. 檢定與穩定性

- 不只看一段：使用 rolling window（例：過去 2 年）反覆算 CCF，看領先關係是否穩定存在。
- 可以搭配 Granger causality test 等進一步判斷 X 的過去值對 Y 是否有額外解釋力。

TradingView 的 Cross-Correlation Lead/Lag Analyzer，就是做 rolling cross-correlation 在多個時間 offset，幫你顯示在某段期間內哪個 lag 的相關度最高，本質上就是上面這套流程圖像化。^{[31] [30]}

4. TA 指標本身的領先 / 落後屬性

再把這件事套回你前面問的 TA-Lib 技術指標：

- 典型「落後指標」
 - 各種均線、MACD、布林帶、ADX、ATR 等，都是用歷史價計算的平滑值，訊號幾乎必然落後價格轉折。^{[32] [33]}
- 典型「領先 / 震盪指標」
 - RSI、KD、CCI、STOCHRSI 等，有時會提早轉頭，被稱為「領先指標」，但在統計意義上，是否領先要靠前述 cross-correlation 或 lead/lag 分析檢驗，而不是憑名稱。^[32]
- 很多經濟指標、技術指標「有時領先、有時落後」

- 在某段景氣循環、某種 regime 下可能有領先性，換到另一段就不一定；這是經濟研究與台股實證都常見的結論。 [34] [25]

實務上做法：把你關心的技術指標（例如 RSI、MACD）、大盤、產業指數、期貨、匯率、利率等等全部丟進同一個 lead/lag 分析框架，讓資料說話。

5. 實際「怎麼做」的建議流程

以你會用 Python / TA-Lib 為前提，整個 lead/lag 研究可以這樣設計：

1. 資料準備

- 選兩個目標序列：例
 - A = 台指期收盤報酬、B = 電子指數報酬
 - A = 大盤報酬、B = MACD 或 RSI 變化
- 轉成等間隔（daily / 5min），補齊假日、同步對齊時間。

2. 統計 lead/lag 分析

- 設一個最大 lag K （例如 20 個交易日），計算各 $k \in [-K, K]$ 的相關係數。
- 畫出 cross-correlation 圖，看 peak 在哪裡。 [26]
- 用 rolling window 做時間穩定性檢查。

3. 導出「可交易規則」

- 若發現「X 領先 Y 2-3 根 K 線且相關度高」，可以測：
 - 當 X 出現特定事件（突破、均值反轉等），Y 是否在接下來 2-3 根出現有統計優勢的方向報酬。
- 用回測框架測試策略收益、風險、穩定性，看是否具備實際交易價值。

4. 視覺化

- 疊圖：原始兩條曲線 + 移位版本。
- 交叉相關圖：lag vs $\rho(k)$ ，加上顯著門檻線。
- 時間熱圖：rolling window $\times$ lag $\rightarrow$ correlation 熱力圖，可以看 regime change。

如果你願意，我可以下一步直接幫你設計一個 Python 模組介面，例如：

```
analyze_lead_lag(series_x, series_y,
                  max_lag=20,
                  window=252,
                  freq='D')
```

輸出：

- 全樣本 cross-correlation 序列
- rolling cross-correlation 矩陣
- 自動偵測「最可能的領先/落後 lag 區間」及其穩定性評分

你就能直接把任何 TA-Lib 指標、價量資料丟進去，系統化檢查誰對誰有穩定領先性。

森

BACK TESTING用法及最強20功能

Backtesting 不只是「把策略丟進歷史資料跑一遍」，而是一整套流程與工具組；下面先講用法流程，再列出我認為實務上最關鍵的 20 個功能。^{[41] [42]}

一、Backtesting 的核心用法流程

1. 明確定義策略規則

- 進出場條件、停損停利、加減碼、交易資產與時間頻率。
- 包含倉位 sizing、最大風險、持倉上限等。^{[43] [41]}

2. 準備歷史資料與交易成本模型

- 乾淨資料：處理除權息、合約換月、存活者偏誤。
- 成本：手續費、點差、滑價（可用常數或流動性函數建模）。^{[44] [45] [46]}

3. 在訓練樣本上跑回測

- 得到收益曲線、風險指標、交易統計。
- 先用「簡單、可解釋」的策略版本過濾掉完全不行的概念。^{[42] [41]}

4. 在樣本外與 Walk-Forward 驗證

- 做一次或多次 out-of-sample 測試。
- 進階：使用 walk-forward（滾動優化 + 滾動驗證），拼接所有 out-of-sample 成為一條「模擬真實」的權益曲線。^{[47] [41] [42]}

5. 最後才進入 paper trading / 小資金實盤

- 用模擬下單檢驗滑價、成交率、基礎設施問題。^{[47] [43]}

二、Backtesting 系統「最關鍵 20 功能」

我把它分成：執行引擎、風險與成本、驗證方法、績效指標四大塊，每一塊列出實務上非常重要的功能。^{[48] [49] [41] [42]}

A. 執行引擎與資料處理（5 項）

1. 多資產、多時間框架支援

- 同時處理多標的（股票、期貨、ETF）、甚至多 timeframe（例如日線 + 5 分線信號）。^{[41] [48]}

2. 事件驅動 / bar 驅動的模擬撮合

- 明確規則：在一根 bar 的哪個時點根據哪個價成交（open/close/mid/高低隨機等），避免 look-ahead bias。^{[42] [41]}

3. 嚴格避免未來函數 (look-ahead) 與資料洩漏

- 所有指標與決策只能用「當下以前」的資料，樣本切割清楚。 [49] [42]

4. 生存者偏誤與下市處理

- 股票回測時必須包含退市標的、避免僅用「活到現在的公司」，否則績效被高估。 [49]

5. 高頻 / 逐筆資料支援 (視需求)

- 若做日內或高頻，能處理 tick / order book 並模擬逐筆成交與排隊情境。 [46] [44]

B. 風險管理與成本模型 (5 項)

6. 彈性倉位管理與資金曲線驅動

- 支援百分比風險、固定金額、Kelly 變形、波動度調整倉位等規則。 [48] [41]

7. 真實成本：手續費、稅、點差、滑價模型

- 可設定固定或動態 (如點差 = 流動性函數、滑價 = 成交量分位數函數)。 [45] [44] [46]

8. 流動性與成交率建模

- 大單分拆、日均量限制、成交量比例限制 (例如不超過當日量 5%)。 [44] [45]

9. 保證金、槓桿與強制平倉規則

- 對期貨、槓桿 ETF 等考慮保證金率、維持保證金、追加保證金場景。 [42]

10. 風險控制規則內建

- 單筆最大虧損、單標的最大曝險、整體最大回撤門檻 (如 >20% 停止策略)。 [50] [48]

C. 驗證與防過度最佳化功能 (5 項)

11. In-sample / Out-of-sample 分割

- 清楚設定訓練與測試期間，開發過程只在 in-sample 做調參。 [41] [49]

12. Walk-forward optimization (滾動最佳化)

- 滾動窗口：在 window1 最佳化，在 window2 測試，然後往前滑動；最後拼接所有測試期結果。 [51] [47] [49] [42]

13. 多策略 / 多參數集的穩健性分析

- 例如：同一策略在不同參數網格上的績效熱圖，看是否只有一小塊「針尖」區域好，還是大片區域都穩健。 [51] [49] [41]

14. 防止「多次測試偏誤」的統計工具

- 如 Deflated Sharpe Ratio、Probabilistic Sharpe Ratio、最小所需樣本長度等。 [49]

15. 前行 / 模擬實盤測試 (Forward / Paper testing) 整合

- 讓策略可以在實時資料上以「虛擬下單」跑一段時間，檢驗回測假設是否成立。 [43] [47]

D. 績效度量與診斷 (5 項)

16. 完整風報指標組

- 年化報酬、總報酬、Sharpe、Sortino、Calmar / MAR、最大回撤、波動率等。 [52] [50] [48]

17. 交易層級統計

- 勝率、盈虧比、平均持有時間、交易頻率、連續虧損次數、各標的貢獻度。 [52] [48] [41]

18. Profit Factor、R-squared 等補充指標

- Profit Factor (總獲利 / 總虧損)、回測權益曲線對基準的 R-squared，檢查是否只是 Beta 曝險。 [50] [48] [52]

19. 穩定性與 regime 分析

- 按年度 / 月度 / 市場環境 (多頭、空頭、盤整) 分段績效，評估策略在不同 regime 的表現。 [48] [50]

20. 輸出可視化與報告

- 權益曲線、回撤曲線、月度熱力圖、rolling Sharpe、因子暴露變化等，便於對策略「直觀體感 + 量化診斷」。 [50] [52] [41]

三、給你的一個實作建議

以你的技術背景，建議把 backtesting 系統抽象成三層：

- Data & Execution layer：資料、撮合、成本、滑價、倉位引擎。
- Strategy layer：純函數式 `on_bar(state) -> orders`，乾淨可測試。
- Analytics & Validation layer：walk-forward、統計檢驗、績效報告。

如果你希望，我可以下一步直接幫你草擬一個 Python backtest 核心框架的介面設計 (class 結構 + 主要方法)，配合你前面 TA-Lib 指標模組與 lead/lag 分析，變成一個可以「指標因子 → 策略規則 → 回測 → walk-forward」的一致流水線。



Common backtesting pitfalls in TA-LIB

When you backtest TA-Lib-based strategies, 大坑幾乎都不是 TA-Lib 本身的數學錯，而是「怎麼用」才會出問題；這裡聚焦在 technical indicator/TA-Lib 情境下最常見的陷阱與對應作法。 [56] [57] [58] [59]

1. Look-ahead bias：指標用到「未來資料」

常見情況：

- 用收盤價算指標，卻在同一根 K 線的收盤價下單
 - 例：用 `close` 計算 RSI 或 MACD，然後在同一根 bar 的收盤價做交易，等於偷看了本 bar 的全部資訊。 [60] [56]
- 指標 shift 錯位

- 在 Pandas 把指標對齊時沒有 `shift(1)`，導致用到當期或未來值。 [56]
- 特例：TA-Lib 某些複雜 matype 實作 bug
 - GitHub 上有人回報 BBANDS 搭配 matype=MAMA 時出現疑似 look-ahead 行為，讓回測績效異常漂亮，移除後就崩掉。 [61] [62]

避免方式：

- 嚴格規則：
 - 所有決策都用「上一根 bar 結束時計算出的指標」：程式上就是 `signal = indicator.shift(1)`。
- 用 event-driven/backtest framework，明確區分「收盤計算指標 → 下一 bar 開盤才可下單」。 [63] [59]
- 對非常特殊的 matype（如 MAMA）保持懷疑，必要時自己驗證 TA-Lib 計算是否只用過去輸入。 [62]

2. Overfitting & indicator mining：指標 + 參數挖太兇

技術指標特別容易落入「亂試組合直到漂亮為止」：

- 對一堆 TA-Lib 指標做無節制 brute-force
 - 隨機挑數十個指標（RSI, MACD, CCI, STOCH, ADX...）+ 多組參數組合，最後只挑報酬最高那組。 [64] [59] [56]
- 高度複雜規則
 - 同時疊加過多條件（例如 10 個指標交叉）讓樣本外完全失效。 [57] [65] [66]

避免方式：

- 把 TA-Lib 指標視為「有經濟直覺的 feature」，先從少數核心開始（例如趨勢 + 動能 + 波動）。
- 明確切分 in-sample / out-of-sample，所有調參只在 in-sample，用 OOS 嚴格驗證。 [58] [59]
- 對參數做「穩健性檢查」：看一片區域都還行，而不是只有單一點 super optimal。 [66] [58]
- 接上 walk-forward optimization，讓每段 OOS 都是「滾動真實」的表現。 [67] [68]

3. Survivorship bias：只用現在還在的標的

用 TA-Lib 在現在的成分股上跑歷史回測，如果資料來源沒包含下市公司，績效會被嚴重高估。 [57] [58]

- 問題：
 - 指標本身沒錯，但你只在「最後活下來的股票」上算 RSI/MACD，再回測 15 年，看起來策略穩賺。
- 避免方式：
 - 使用不含存活者偏誤的歷史成分與報價資料（包含退市標的）。 [58]
 - 多標的策略時，用「當時可投資宇宙」建構回測，而不是用今天的 universe 倒推。

4. 忽略成本、滑價與流動性

TA-Lib 指標多在 bar 資料上計算，容易讓人忘記 microstructure：

- 忽略手續費、稅、點差、滑價
 - 短線指標策略（例如 5 分 RSI、均線交叉）不加成本時，看起來超神；實盤完全不同。 [69] [70] [56]
- 完全不考慮成交量與深度
 - 用 TA-Lib 在小型股 1 分線上做高頻進出，假設每一筆都能以 close 價全額成交。 [69] [57]

避免方式：

- 建立 realistic cost model：
 - 固定手續費 + 交易稅 + 點差的一定比例滑價（可隨波動或量調整）。 [70] [69]
- 流動性約束：
 - 單日單股成交量上限（例如不超過當日成交量 3–5%）。 [71] [69]

5. 指標多重共線性與「假多樣性」

TA-Lib 裡有 150+ 指標，但很多是彼此高度相關的變形，例如不同 period 的 SMA/EMA、MACD vs PPO、RSI vs STOCHRSI 等。 [57]

- 問題：
 - 以為用了「很多 feature」其實只是堆同一類訊號，容易 overfit 而不增加真實資訊量。 [57]
- 避免方式：
 - 用相關矩陣或 PCA 檢查指標之間的共線性，刪掉高度重疊者。 [57]
 - 在策略層面，用少數代表性指標，而不是所有 variant 一次上。

6. 時間粒度與觸發點處理不一致

搭配 TA-Lib 指標時，常見錯誤是「圖上看的是日線指標，交易卻用到日內細節」但沒在回測重現：

- 例如：
 - 用日線 BBANDS 判斷突破，但實際交易用 5 分線掛單，回測卻假設可以在當日極佳價位成交。
- 避免方式：
 - 要嘛全程日線邏輯（信號與成交都在日線 open/close）。
 - 要嘛 multi-timeframe backtest，把日線指標 downsample 成 intraday rule，並用 intraday 資料模擬實際可成交區間。 [59] [63]

7. Regime change：指標只在特定行情有用

技術指標的大部分假設（趨勢、均值回歸、波動結構）都隱含某種市場 regime；單一長期 backtest 很容易把不同 regime 混在一起，誤以為策略「長期有效」。^{[56] [58]}

- 問題：
 - 例如 MA crossover 在強趨勢年很亮眼，在盤整年大虧；整體看起來小賺，但真實執行時心理壓力巨大。
- 避免方式：
 - 按市場狀態分段看績效：牛市、熊市、震盪市。^{[72] [73]}
 - 把 regime 作為顯式 feature（如用 ADX 判斷是否啟用趨勢策略）。

8. 「完美回測 = 問題策略」的錯覺

在 TA 指標策略中，出現「幾乎無回撤、勝率極高」的 equity curve，往往代表嚴重過度最佳化或 bias。^{[65] [64] [56]}

- 特徵：
 - 超平滑權益曲線、幾乎每年都大賺、幾乎沒有大 drawdown。^[64]
 - 極度依賴少數幾筆大型獲利交易。^[56]
- 檢查：
 - 換資料期間、換市場（例如從台股換成美股或其他產業指數），看策略是否崩壞。^{[59] [58]}
 - 做參數敏感度分析；看稍微變動 RSI 週期或 MA 週期後，績效是否立刻從神變爛。^{[66] [58]}

簡單實務 Checklist（專門給 TA-Lib 回測）

每次你用 TA-Lib 做 backtest，可以檢查：

1. 所有指標值是否在下單時已向右 shift(1)，拒絕任何未來資料。
2. 僅使用少數核心指標，且有經濟/行為直覺支撐。
3. 有 in-sample / out-of-sample，且做過 walk-forward 或至少外樣 OOS 測試。
4. 使用無存活者偏誤且含退市標的的歷史資料。
5. 有建模手續費、稅、點差、滑價與流動性限制。
6. 用 regime 分段看策略在不同行情型態的表現，而不是只看整體。

如果你願意，我可以幫你寫一個「TA-Lib backtest safety wrapper」設計草稿：例如一個 SafeIndicator / SafeSignal 介面，強制所有輸出自動 shift、並在 backtest 開始前做幾項 sanity check（如未來資料檢測、參數 grid 熱圖、子期間績效報表）。

泡沫定義 過熱定義 超跌定義 經得過實測嗎

在嚴格的學術與量化語境裡，「泡沫」「過熱」「超跌」都有工作定義，但沒有任何一個是「萬用準確」的；能不能經得起實測，關鍵在你怎麼定義和驗證。下面分別說明。^{[78] [79] [80] [81] [82] [83]}

泡沫 (Bubble) 的定義與實證

常用的現代定義是：資產價格長期顯著高於其基本面價值，這段偏離稱為泡沫成分 $B_t = P_t - V_t$ 。^{[84] [82] [78]}

- 基本面價值 V ：未來現金流折現的期望值（在給定資訊下的合理估值）。^{[82] [84]}
- 泡沫：當市場價格 P 長期、大幅高於這個 V 時，就被稱為 asset price bubble。^{[85] [78] [82]}

是否「經得過實測」：

- 實證方法很多，例如 Phillips–Shi–Yu (PSY) 類爆炸性單根檢定、利用多重檢定演算法做即時泡沫監測，對 S&P 500 等資料，在 1990–2020 年間能偵測出網路泡沫、金融海嘯等事件，樣本內表現不錯。^[86]
- 也有利用橫斷面偏度、找出「合理回報」當作基準，再把超出部分視為泡沫成分的方法，實測能對 DJIA 中多次泡沫期間給出信號。^[87]

限制在於：

- 真實的「基本面價值」不可觀，只能用模型近似，結論依賴假設。^{[84] [82]}
- 實務上通常是「事後」比較容易證明泡沫存在，事前偵測仍然有誤報與漏報風險。^{[86] [87]}

過熱 (Overheating / Overbought) 的定義與實證

這裡有兩個層次：宏觀經濟過熱 vs 市場或個股價格過熱。

1. 宏觀「經濟過熱」

- 定義：經濟成長過快，需求長期大幅超過供給，導致顯著通膨、產能緊繃、勞動市場過度吃緊與資產價格泡沫。^{[88] [89] [90]}
- 特徵：物價快速上漲、失業率極低、工資與信貸擴張、資產價格飆升等。^{[90] [88]}

2. 金融市場的「價格過熱 / overbought」

- 技術面：價格漲幅、動能遠超 fundamentals 合理範圍；常用 $RSI > 70$ 、 $Stochastic > 80$ 等作為 overbought 條件。^[83]
- 基本面：估值指標 (P/E 、 P/B 、 $CAPE$ 、yield spread 等) 顯著高於歷史分佈與基本面支撐。^{[89] [85]}

是否「經得過實測」：

- 宏觀上，很多研究表明當一連串「過熱」訊號同時出現（高通膨、低失業、信貸暴增）時，之後經濟放緩或危機的機率顯著上升，屬於統計上有用的預警，但精準 timing 仍不可靠。^{[88] [90]}
- 技術面 overbought（例如 $RSI > 70$ ）在實證上往往在強趨勢中可以「長期過熱而不跌」，單獨使用作反轉訊號，預測力有限；搭配趨勢 regime、風險因子後，效果較為穩健，但依市場與期間而異。

超跌 (Oversold / Overshooting) 的定義與實證

「超跌」本質上是「跌到低於合理價值」，對應 oversold。[81] [80]

- 基本定義：價格跌破合理或基本面價值，出現明顯低估，多由恐慌性賣壓、羊群行為造成。[80] [81]
- 技術指標：
 - RSI < 30、Stochastic < 20、價跌遠低於移動平均或歷史波動帶。[81] [83] [80]
- 基本面：
 - 價格低於帳面價值 (P/B < 1)、估值落在長期歷史分佈極低分位 (例如 P/E 在過去 20 年的 5% 分位數以下)。[80] [81]

是否「經得過實測」：

- 多個實證與教學資料指出，單純以「技術 oversold 指標」作為反轉買點，成功率依市場與期間不同，往往不足以形成 alpha，容易在趨勢崩跌中反覆被「超跌再更跌」。[83] [80]
- 結合基本面低估與技術 oversold，作為 value + sentiment/flow 策略的一環，長期實證普遍顯示有一定超額報酬，但需承受長時間低估與深度回撤風險。[81] [80]

統一回答：「經得過實測嗎？」

簡化說：

- 泡沫 (bubble)
 - 有嚴格數理定義與多種統計檢定，對歷史資料 (股市、房市) 可以事後很好地辨識出泡沫階段，且與重大事件相符。[78] [87] [82] [86]
 - 但即時、事前辨識仍有不小不確定性，更多是「風險預警」而非精準 timing 工具。
- 過熱 (overheating / overbought)
 - 宏觀過熱指標 (高通膨、低失業、信貸擴張等) 在實證上對未來放緩或危機風險有預測力，但時點不精準。[90] [88]
 - 技術過熱 (RSI > 70 等) 單獨對短期反轉的預測力有限，若不結合趨勢 regime 或其他因子，策略容易失效。[83]
- 超跌 (oversold / overshooting)
 - 單靠技術 oversold 指標 (RSI < 30 等) 很難穩定賺錢；
 - 結合估值低估與情緒極端的「超跌」概念，在長期 value / contrarian 策略中，有不錯的實證支持，但回撤與等待成本高。[80] [81] [83]

換句話說：

- 這三個概念在學術上都有可操作的定義，也有不少實證支持「有用」，但都不是穩定的精準轉折預測器，而是「風險與機會區間」的標籤。
- 要讓它們經得起實測，你得：

- 先選明確、可量化的定義（技術指標門檻、估值分位數、泡沫檢定等）；
- 再把它們放進嚴謹的回測框架裡（含成本、風控、樣本外測試），看是否在你關注的市場與頻率上有穩定 edge。

如果你願意，我可以幫你設計一套「泡沫/過熱/超跌量化標籤 + 回測框架」的具體規格，例如：

- 日線層級：RSI / Bollinger band / return quantile；
- 估值層級：歷史分位數 + cross-section 異常值；
- 加上事件研究或 regime 策略回測，直接在台股或你關注的標的上檢驗這些區間標籤是否真的有 alpha。

✱

過熱市場的TA-LIB判斷標準及回測

過熱市場用 TA-Lib，通常是先「定義過熱條件」，再放進嚴謹回測框架檢驗有沒有 edge；下面用你熟悉的 quant 角度整理一套相對標準化的做法。[\[93\]](#) [\[94\]](#) [\[95\]](#) [\[96\]](#) [\[97\]](#)

一、常見的「過熱」技術標準（TA-Lib 可直接算）

以 TA-Lib 可取得的指標來說，市場過熱（overbought/overextended）常見量化條件有：

- RSI (`talib.RSI`)
 - 經典門檻：RSI > 70 視為 overbought，RSI < 30 視為 oversold。[\[94\]](#) [\[96\]](#) [\[93\]](#)
 - 強趨勢調整：多頭時可調成 80/20，甚至 85/15，避免太早逆勢。[\[95\]](#) [\[96\]](#) [\[93\]](#)
- Bollinger Bands (`talib.BBANDS`)
 - 價格收盤價高於上軌（中軌 = 20MA，上下軌 = $\pm 2\sigma$ ）視為價格「向上過度延伸」。[\[98\]](#) [\[99\]](#)
 - 若同時標準差擴大，代表波動加劇的過熱放大。
- 均線乖離（可自行用 SMA + 計算）
 - 乖離率 = (收盤價 - N 日 MA) / N 日 MA。
 - 當乖離率高於歷史分佈高分位（如 95% 分位數），可標記為過熱區。
- Stochastic / StochRSI (`STOCH` / `STOCHRSI`)
 - %K 或 %D 高於 80 常視為 overbought 區，上方久掛代表短期價位極端。[\[100\]](#)
- 複合條件（實務上很常見）
 - 如：RSI > 70 且 收盤價 > 上軌 → 強過熱；
 - 或：RSI > 80 且 20 日乖離率 > 歷史 90% 分位。[\[101\]](#) [\[102\]](#) [\[103\]](#) [\[99\]](#) [\[98\]](#)

這些都能用 TA-Lib 直接計算，再把條件標記成布林欄位 `is_overheated`。

二、定義「過熱市場」的量化標籤

比較嚴謹的做法是把過熱當成一個 regime / state，不是單點訊號：

- 單日標籤 (bar-level)
 - 例：`is_overheated = (RSI > 75) & (close > upper_band)`。[\[99\]](#) [\[93\]](#)
- 區間標籤 (regime-level)
 - 用連續 N 天有高比例 `is_overheated` 或指標長期在高檔，例如：
 - 最近 10 日有 7 日 `RSI > 70`；
 - 最近 20 日有超過一半時間收盤價在 1σ 以上。

然後你可以在 DataFrame 裡加一欄 `regime = 'hot' / 'normal'` 作為後續切分與策略條件。

三、如何用 TA-Lib 定義過熱並做回測（實作流程）

下面是一套你可以直接落到程式的流程，框架不綁特定語言；TA-Lib 負責特徵，backtester 負責交易邏輯與績效評估。[\[104\]](#) [\[97\]](#) [\[105\]](#)

1. 資料與指標準備

- 取得標的的 OHLCV 日資料（或你要的頻率）。
- 用 TA-Lib 計算：
 - `RSI(close, timeperiod=14)`
 - `BBANDS(close, timeperiod=20, nbdevup=2, nbdevdn=2)`
 - 自算 `bias_20 = (close - SMA20) / SMA20` 等。

2. 標記過熱條件 (signal layer)

- 例一（保守版）：
 - `overheated = (RSI > 70) & (close > upper)`。[\[96\]](#) [\[93\]](#) [\[99\]](#)
- 例二（強趨勢調整）：
 - `overheated = (RSI > 80) & (close > upper)`。[\[93\]](#) [\[96\]](#)
- 例三（乖離極端）：
 - 先計算乖離率的歷史分位數，過去 N 年中 $> 95\%$ 分位的日子標記為過熱。

3. 回測邏輯設計

- 反轉型策略 (mean reversion)：
 - 做空為主：當出現 `overheated`，下一根 bar 以 open 做空，平倉條件可設定：
 - 價格回到中軌附近，或
 - RSI 回落到 50–60 區間。[\[102\]](#) [\[103\]](#) [\[101\]](#)
- 趨勢追蹤強化 (trend continuation)：
 - 不是立刻做空，而是把過熱當成「趨勢很強」標籤，用於：
 - 暫停加倉、提高移動停損 (lock in profit)；

- 只在過熱消失且出現明顯反轉訊號時才反手。

4. 嚴謹回測細節（避免坑）

- 訊號必須用 `shift(1)` :
 - 信號在第 t 天收盤生成，交易在第 $t+1$ 天 open 執行，避免 look-ahead。
- 加上手續費、交易稅、點差與滑價；
- 設定持倉限制與風控（單筆風險、整體回撤）。[\[97\]](#) [\[105\]](#) [\[106\]](#) [\[107\]](#)

5. 驗證過熱標準是否有 edge

- 對比基準：
 - 單純 buy & hold ；
 - 單純趨勢策略（如 MA crossover）+ 無過熱 filter。
- 分 regime 評估：
 - 牛市、熊市、盤整時，過熱反轉策略的勝率和期望是否差很多。
- 參數穩健性：
 - 場景測試 RSI 門檻 70/75/80、BBands $\sigma=1.5/2/2.5$ 、不同 holding rule，看是否有「一整片區」表現良好，而非只是一個針尖點。[\[95\]](#) [\[96\]](#)

四、實證上過熱條件的典型結果（文獻與實務觀察）

從公開研究與各種策略分享來看（股票、外匯、加密貨幣）：

- 單純用「RSI > 70 就做空」在長期與強趨勢市場表現通常不佳，因為 RSI 可以在 70–90 「黏很久」。[\[94\]](#) [\[96\]](#) [\[93\]](#) [\[95\]](#)
- 把 RSI 與 Bollinger Bands 或價格乖離一起使用，做 mean reversion 回測，在橫盤或均值回歸較強的市場，會有一定勝率和正期望，但在大牛市或大熊市中仍容易被趨勢「碾過」。[\[103\]](#) [\[101\]](#) [\[98\]](#) [\[100\]](#) [\[102\]](#) [\[99\]](#)
- 很多系統把「過熱」條件用作風控與減倉而非強制反向進場，實證上較穩健：例如，在過熱 regime 裡提高停利與停損嚴格度，而不是硬空頂。

五、若你要在台股 / 你自己的框架做

建議你這樣落地：

1. 把「過熱」定義解耦為一個獨立模組 `tag_overheated(df)` :
 - 內部用 TA-Lib 算 RSI、BBands、bias，輸出 `overheated_level (0,1,2)` 。
2. 在策略層，只使用這個 tag :
 - `if overheated_level >= 1: adjust_position()`
 - 或 `if overheated_level >= 2 and trend_regime == range: open_short()`。
3. 用你現有 backtest engine 做：
 - 全市場、長樣本、分產業、分市值層級的回測，統計在過熱日開倉與非過熱日開倉的風報差異。

如果你要，我可以直接幫你擬一個 Python/TA-Lib 的 pseudo-code：

- 一個函式專門標記「過熱市場」；
- 一個簡單 mean-reversion 策略範本；
- 一個測試模組去自動產出「過熱 vs 非過熱」的分組績效報表。

✱

回測工具 TOP 20 LOCAL FREE LIBS

下面列的是「可在本機用、免費、開源或免費授權」的 backtesting / 量化庫，偏重你應該實際會用到的 Python + 少量 R/C++，並用表格整理。[\[111\]](#) [\[112\]](#) [\[113\]](#) [\[114\]](#) [\[115\]](#) [\[116\]](#) [\[117\]](#) [\[118\]](#) [\[119\]](#)

Python 生態系（主要推薦）

#	Library	語言	授權/ 費用	特點摘要
1	Backtrader	Python	GPLv3， 免費	功能最完整之一，多資產、多時間框架、實盤整合、社群大。 [113] [120]
2	backtesting.py	Python	MIT， 免費	API 精簡，上手快，內建優化與圖表，適合策略原型。 [111] [112] [114]
3	Zipline	Python	Apache 2.0	Quantopian 開源引擎，事件驅動，對美股日線回測成熟。 [117] [119]
4	bt	Python	MIT	偏投組/因子回測，基於 pandas，適合多資產資產配置策略。 [117]
5	PyAlgoTrade	Python	Apache 2.0	老牌事件驅動框架，含技術指標、事件源整合（Yahoo 等）。 [117] [119]
6	QSTrader	Python	MIT	QuantStart 推出，設計強調模組化、機構級結構。 [117] [119]
7	vectorbt / vectorbt-next	Python	MIT	向量化回測，極快，適合做大規模參數掃描與因子實驗。 [119]
8	fastquant	Python	MIT	封裝 Backtrader 等，提供簡化 API，快速測策略 idea。 [119]
9	pysystemtrade	Python	BSD	Rob Carver 的系統交易工具箱，偏期貨/CTA，多策略組合。 [117]
10	PortfolioBacktester 類工具 (自建 + PerformanceAnalytics)	Python	多為開源	常見用 pandas + numpy + 自寫 loop + statsmodels，搭配 pyfolio / empyrical 類工具做績效分析（你應該已在走這條路）。 [119]

(9、10 類比較偏框架/工具集而不是單一成品，但對你這種要客製化的人很適合。)

R 生態系（量化/投組向）

#	Library	語言	授權/費用	特點摘要
11	quantstrat	R	GPL	經典技術/事件回測框架，與 blotter / TTR / quantmod 整合。
12	PerformanceAnalytics	R	GPL	不是回測引擎，但提供全套績效與風險分析函數。
13	portfolioBacktest	R	GPL-3	多投組、多資料集自動回測，重點在投組比較與報告。 ^{[116] [121]}
14	backtest (CRAN)	R	GPL	針對因子/投組回測，支援分桶、分組回報與圖表。 ^[118]

其他語言／專門引擎

#	Library / Engine	語言	授權/費用	特點摘要
15	mccaffers/backtesting-engine-cpp	C++	MIT	高效能 C++ 回測引擎雛型，可處理 tick 資料 + 多策略並行。 ^[115]
16	Lean Engine (QuantConnect 開源部分)	C# / Python	Apache 2.0	雖然雲端商業平台，但核心引擎可本機跑，支援多市場。
17	TradeLink / OpenQuant 類舊框架	C#/C++	多為免費	老牌本地回測 + 下單框架，有一定學習成本。
18	Amibroker trial + AFL (非開源但可本機)	專用語言	收費，有試用	嚴格來說不是「lib」，但本機回測極快、策略語言簡潔。
19	R + data.table + 自寫引擎	R	開源	很多 paper 級 backtest 直接用 data.table/xts 手刻 loop。
20	Julia + MarketData/TimeSeries 庫	Julia	MIT	若你之後玩 Julia，可自建向量化 backtest，性能佳。

你若鎖定「Python、本機、免費、TA-Lib 友善」，我會優先推薦：

- Backtrader：做完整盤中邏輯、多資產、期貨/槓桿等。^{[113] [120]}
- [backtesting.py](#)：快速原型測試 + 教學／展示。^{[112] [114] [111]}
- vectorbt：做大量參數掃描、因子/指標組合的 brute-force 實驗。^[119]
- QSTrader：如果你要自己搭建「研究平台級」結構。^{[117] [119]}

如果你告訴我你現在的技術棧（純 Python + pandas + TA-Lib？有沒有需求跑 TWSE/TPEX 多標的日線/分鐘？）我可以幫你挑 2-3 個最適合的，順便給一個「指標模組 → backtest → walk-forward」的一致架構示意。

✻

若有雙軸圖顯示商品的風險因子你會選甚麼??

雙軸圖如果是「一眼看商品風險輪廓」，我會優先選一個「波動/系統風險」 + 一個「尾部/回撤風險」的組合，讓圖既直觀又有資產定價含意。^{[126] [127] [128] [129] [130] [131] [132]}

推薦的幾組雙軸搭配

1. 橫軸：年化波動率，縱軸：最大回撤

- 波動率 (std of returns) 是經典風險度量，與 beta 等系統風險高度相關。 [128] [129] [126]
- 最大回撤 directly 告訴使用者「最慘會跌多深」，補足均方風險看不到的 tail risk。 [131] [132]
→ 解讀：右上角 = 高波動 + 深回撤，左下角 = 穩定低風險商品。

2. 橫軸：Beta（相對市場風險），縱軸：最大回撤 或 年化波動

- Beta 是系統風險暴露，來自 CAPM / 多因子文獻。 [127] [129] [128]
- 縱軸放回撤（或波動）可以看出「同樣 beta 下，誰風險控得比較好」。 [132] [131]
→ 解讀：同一個 beta 水平上，回撤較小的商品風險控制較佳。

3. 橫軸：違約/損失機率（或 drawdown 發生頻率），縱軸：損失幅度

- 這其實就是金融版的 risk map：x = 發生機率，y = 影響程度。 [130] [133]
→ 解讀：右上角 = 高機率 + 高損失，風險重災區；左下角 = 可接受區。

4. 若要「風險因子」視角（因子投資）

- 橫軸：市場因子暴露 (beta)，縱軸：某一 style factor beta (如 value/momentum/quality)。 [134] [135] [136] [137] [127]
- 可用顏色或點大小編碼年化波動或回撤。
→ 解讀：可以看一檔商品是「高 beta 高 momentum」還是「低 beta 高 quality」之類的風險型態。

實作建議（雙軸圖）

- 單個商品：畫在「多商品散點圖」裡，用顏色/大小再帶一個維度（如夏普或久期）。 [133] [130]
- 多商品組合比較：
 - 同一張圖上標出：標的、基準指數、你自己的策略組合。
 - 把 benchmark 標成十字或特別顏色，方便看「向哪個方向是更佳風險表現」。

如果你跟我說具體情境（例如：台股 ETF 給一般投資人、還是你自家 CTA 策略給專業客戶），我可以幫你精準選一組 x/y + 顏色編碼，順便給一個 pandas/Plotly 的實作框架。

✻

這些風險因子的TA-LIB計算函數有哪些

如果目標是把「商品風險因子」接到 TA-Lib，對應函數大致可以這樣選：波動 / 系統風險用 TA-Lib，回撤等需要自己算。

1. 波動類風險（Volatility risk）

這部分 TA-Lib 有完整函數組。 [141] [142] [143] [144]

- TRANGE(high, low, close)
真實波幅（單期 range），可再自行做 rolling 統計。

- ATR(high, low, close, timeperiod=14)
平均真實波幅，經典波動風險度量。 [142] [143] [144] [141]
- NATR(high, low, close, timeperiod=14)
標準化 ATR (除以 close，變成百分比)，適合跨商品比較。 [143] [144] [141] [142]
- STDDEV(real, timeperiod, nbdev=1)
直接算報酬或價格的標準差，用來當年化波動的基礎。 [145] [146] [147]
- VAR(real, timeperiod, nbdev=1)
變異數，如果你要做風險模型或 CAPM 類計算。 [146] [147] [145]

2. 系統風險與相關性 (Beta / correlation)

這部分在 TA-Lib 的 Statistic Functions。 [147] [145] [146]

- BETA(real0, real1, timeperiod=5)
計算資產相對基準（如大盤）的 beta，衡量系統風險暴露。 [145] [146] [147]
- CORREL(real0, real1, timeperiod=30)
皮爾森相關係數，看跟市場或其他資產的聯動風險。 [146] [147] [145]
- LINEARREG / LINEARREG_SLOPE / LINEARREG_ANGLE
線性回歸與斜率，可當趨勢強度或 beta 類 proxy。 [147] [145] [146]

3. 其他可當風險 proxy 的 TA-Lib 函數

雖然不是「風險函數組」，但常被用來描述風險狀態：

- ADX 系列（趨勢強度，trend risk proxy）
- OBV、MFI（量價風險、流動性壓力 proxy）

這些在 TA-Lib 對應 Momentum / Volume 群組，你前面已經有列表可接。 [148] [149] [150]

4. 需要自己算的部分 (TA-Lib 沒有)

- 最大回撤、回撤曲線、Calmar 比率
- Value at Risk (VaR)、Expected Shortfall (CVaR)
- 高階 tail risk（偏度、峰度）

這些一般用 pandas/numpy + 回測結果自己算，或交給 pyfolio / empyrical 之類的分析庫處理。 [151]

實務上，若你要畫「雙軸風險圖」而又想最大化 TA-Lib 再利用，最直覺的搭配是：

- X 軸：基於 STDDEV 或 NATR/ATR 算出的年化波動。 [141] [145] [146] [147]
- Y 軸：基於回測權益曲線手算的最大回撤（非 TA-Lib），或 beta（用 BETA 算）。 [145] [146] [147]

如果你要，我可以下一步直接給一個 pandas + TA-Lib 的 code skeleton，把「收盤 → 報酬 → ATR / NATR / STDDEV / BETA → 風險 DataFrame」整套產出好，方便你之後直接餵給畫圖或回測模組。

天數是如何決定的

天數本身沒有「正解」，只有「跟你的用途、資料特性、交易週期是否匹配」；以你剛問的 TA-Lib 風險因子來說，可以這樣決定：

1. 先從「策略與決策週期」反推

- 短線（日內、1-5 日持有）：
 - 波動／ATR：常見 10-20 期；
 - Beta / CORREL：20-60 期（約 1-3 個月）。
- 波段（數週到數月）：
 - 波動／ATR：14-60 期；
 - Beta / CORREL：60-252 期（3-12 個月）。
- 長線（半年以上）：
 - 波動／ATR：60-252 期；
 - Beta / CORREL：252-756 期（1-3 年）。

直覺是：「你打算持有多久，就用至少同量級、甚至 2-3 倍的期間估風險」，才不會只看到短期噪音。這種按投資週期選擇窗口長度的做法在風險估計與資產定價研究中是標準做法。^{[156] [157] [158]}

2. 再看「數據性質與穩定性」

- 高波動／regime 常變的標的（小型股、商品、加密）：
 - 窗口太長會把 regime 混在一起，建議用較短的 rolling（例如 ATR 14-30，beta 60-120），並用 rolling 檢查穩定性。^{[159] [160] [161]}
- 穩定大盤、IG 債：
 - 可以用較長期間（ATR 30-60，beta 252），換取估計穩定。^{[162] [156]}

3. 實證微調（不是拍腦袋）

最後，天數應該透過回測／統計檢驗來「驗證」而不是只靠慣例：

- 對多個 timeperiod（例如 10, 14, 20, 30, 60）
 - 比較：
 - 風險估計的穩定度（rolling 標準差的波動）
 - 用這些風險因子做 sizing / 風控後，策略在樣本內與樣本外的績效。
- 選：
 - 不是績效最高那一點，而是「一小段區間都穩定不錯」的區間（穩健性）。

這類「以窗口長度做敏感度分析與穩健性檢驗」是量化資產定價與風險模型中常見的方法。^{[160] [161] [162]}

如果你願意，我可以幫你設計一個小工具：給一組 timeperiod 列表，它自動用 TA-Lib 算 ATR / STDDEV / BETA，然後對比不同期間下的風險估計穩定性與策略回測結果，幫你選出「適合你商品與週期」的一組天數區間。

✻

1. <https://blog.csdn.net/MARY197011111/article/details/79622184>
2. <https://www.finebi.com/blog/article/686e645f28946ecca8c3bf24>
3. <https://www.finlab.tw/python-簡單158種技術指標計算/>
4. https://doc.finlab.tw/details/tech_ind/
5. <https://blog.csdn.net/zhangyunchou2015/article/details/147185690>
6. <https://ithelp.ithome.com.tw/articles/10244303>
7. <https://docs.dolphindb.com/zh/modules/ta/ta.html>
8. https://quantpass.org/python_talib-2/
9. <https://bigquant.com/wiki/doc/VHGjF0ox8k>
10. <https://ta-lib.org/functions/>
11. <https://github.com/mrjbq7/ta-lib/blob/master/docs/func.md>
12. <https://docsinchinese.gitbooks.io/talib/>
13. <https://www.backtrader.com/docu/talibindautoref/>
14. <http://yhhuang1966.blogspot.com/2025/01/python-ta-lib.html>
15. <https://machinelearning-basics.com/what-is-ta-lib-and-how-to-implement-technical-indicators-in-python/>
16. https://blog.csdn.net/weixin_47339916/article/details/147352882
17. <https://github.com/TA-Lib>
18. <http://www.100apple.net/article/detail.html?id=694>
19. <https://ta-lib.org>
20. <https://ta-lib.github.io/ta-lib-python/funcs.html>
21. <https://ta-lib.github.io/ta-lib-python/>
22. <https://github.com/fja05680/TA-Lib-tutorial>
23. <https://blog.csdn.net/fengidea/article/details/142358626>
24. https://blog.csdn.net/weixin_47339916/article/details/147370938
25. <https://rich01.com/what-leading-indicator-lagging-3/>
26. <https://www.fastercapital.com/content/Cross-Correlation--Cross-Correlation--Unveiling-the-Lagged-Relationships-Between-Series.html>
27. <https://www.fe.training/free-resources/financial-markets/leading-lagging-coincident-economic-indicators/>
28. <https://www.etftrends.com/etf-strategist-channel/reading-tea-leaves-leading-lagging-coincident-indicators/>
29. <http://mx.nthu.edu.tw/~ylihuang/pdf/LARSRemi2.pdf>
30. <https://tw.tradingview.com/script/GRHeBYmK-Cross-Correlation-Lead-Lag-Analyzer/>

31. <https://www.tradingview.com/script/GRHeBYmK-Cross-Correlation-Lead-Lag-Analyzer/>
32. <https://www.avatrade.com/education/technical-analysis-indicators-strategies/leading-lagging-indicators>
33. <https://ta-lib.org/functions/>
34. <https://bm.mcut.edu.tw/var/file/46/1046/img/1583/194520258.pdf>
35. <https://www.macromicro.me/charts/56/leading-lagging-indicator>
36. <https://ws.ndc.gov.tw/Download.ashx?u=LzAwMS9hZG1pbmlzdHJhdG9yLzEwL3JlbGZpbGUvNjA5MS8xMTYxMCAwMDAxNjExXzIucGRm&n=MDIj5Y%2Bw54Gj5pmv5rCj6aCY5YWI5Y%2BK5ZCM5pmC5oyH5qiZ5LmL5o6i6KiOiCg5NjggSykucGRm&icon=.pdf>
37. <https://www.pressplay.cc/project/D03782BE5425FA90C4829C846BC8B03A/articles/B544CC6830725A94D3CBCF9F4CD94629>
38. <https://www.oanda.com/bvi-ft/lab-education/dictionary/coincident-indicator/>
39. <https://www.3people.com.tw/知識/經濟觀念與經濟指標-/金融證照-理財規劃人員/8e6b6cc5-8662-48e5-bc5e-dcba0f4ea214>
40. <https://www.bostonifi.com/resources/blog/leading-lagging-indicators>
41. <https://www.quantinsti.com/articles/backtesting-trading/>
42. https://palomar.home.ece.ust.hk/MAFS5310_lectures/slides_backtesting.pdf
43. <https://www.luxalgo.com/blog/backtesting-vs-forward-testing-validating-your-strategy/>
44. <https://www.exegy.com/avoiding-slippage-equities-trading-with-backtesting/>
45. <https://quantra.quantinsti.com/glossary/How-to-Avoid-Common-Mistakes-in-Backtesting>
46. <https://www.luxalgo.com/blog/backtesting-limitations-slippage-and-liquidity-explained/>
47. <https://surmount.ai/blogs/walk-forward-analysis-vs-backtesting-pros-cons-best-practices>
48. <https://www.quantvps.com/blog/guide-to-quantitative-trading-strategies-and-backtesting>
49. <https://starqube.com/backtesting-investment-strategies/>
50. <https://blog.alltick.co/how-to-spot-a-strong-quant-backtest/>
51. <https://github.com/TonyMa1/walk-forward-backtester>
52. <https://www.luxalgo.com/blog/top-7-metrics-for-backtesting-results/>
53. https://www.reddit.com/r/algotrading/comments/1j187b3/my_walkforward_optimization_backtesting_system/
54. <https://tw.tradingview.com/scripts/backtest/>
55. <https://www.fxreplay.com/learn/how-to-optimize-a-trading-strategy-using-backtesting>
56. <https://www.luxalgo.com/blog/backtesting-traps-common-errors-to-avoid/>
57. <https://sites.google.com/view/technicalindicators/backtested-trading-indicators>
58. <https://starqube.com/backtesting-investment-strategies/>
59. <https://www.quantinsti.com/articles/backtesting-trading/>
60. <https://ftmo.com/en/blog/which-are-the-worst-backtesting-mistakes/>
61. <https://github.com/mrjbq7/ta-lib/issues/354>
62. <https://github.com/TA-Lib/ta-lib-python/issues/354>

63. <https://www.luxalgo.com/blog/how-to-build-a-backtesting-engine-in-python/>
64. <https://www.fxreplay.com/learn/why-a-perfect-backtest-often-means-a-flawed-strategy>
65. https://www.reddit.com/r/Traiding/comments/1fnj3hi/backtesting_pitfalls_overfitting_and_realworld/
66. <https://www.truedata.in/blog/mistakes-to-avoid-when-backtesting-your-trading-strategy>
67. <https://surmount.ai/blogs/walk-forward-analysis-vs-backtesting-pros-cons-best-practices>
68. <https://github.com/TonyMa1/walk-forward-backtester>
69. <https://www.exegy.com/avoiding-slippage-equities-trading-with-backtesting/>
70. <https://www.luxalgo.com/blog/backtesting-limitations-slippage-and-liquidity-explained/>
71. <https://quantra.quantinsti.com/glossary/How-to-Avoid-Common-Mistakes-in-Backtesting>
72. <https://blog.alltick.co/how-to-spot-a-strong-quant-backtest/>
73. <https://www.luxalgo.com/blog/top-7-metrics-for-backtesting-results/>
74. <https://blog.csdn.net/HiWangWenBing/article/details/154904816>
75. https://www.reddit.com/r/algotrading/comments/gyvist/is_there_a_downside_of_using_pandas_for/
76. <https://www.socratesplatform.com/blog/avoid-these-common-technical-analysis-mistakes>
77. <https://www.buildalpha.com/backtesting-trading-strategies/>
78. <https://www.chicagofed.org/publications/chicago-fed-letter/2012/november-304>
79. https://en.wikipedia.org/wiki/Economic_bubble
80. <https://www.investopedia.com/terms/o/oversold-bounce.asp>
81. <https://quantra.quantinsti.com/glossary/Oversold>
82. <https://www.econstor.eu/bitstream/10419/313111/1/191781481X.pdf>
83. <https://www.ig.com/en/trading-strategies/understanding-overbought-and-oversold-trading-conditions-241114>
84. <https://arxiv.org/html/2502.10101v1>
85. <https://corporatefinanceinstitute.com/resources/economics/price-bubble/>
86. <https://elischolar.library.yale.edu/cgi/viewcontent.cgi?article=3690&context=cowles-discussion-paper-series>
87. <https://a-e-l.scholasticahq.com/article/124262-testing-for-asset-price-bubbles-an-alternative-approach>
88. <https://longbridge.com/en/learn/overheated-economy-101424>
89. <https://midandwest.co.uk/glossary-overheating>
90. <https://ace-usa.org/blog/research/economic-policy/signs-of-an-overheating-economy/>
91. <https://www.chicagofed.org/~media/publications/chicago-fed-letter/2012/cflnovember2012-304-pdf.pdf>
92. <https://study.com/academy/lesson/asset-price-bubble-definition-model.html>
93. <https://www.luxalgo.com/blog/rsi-overbought-and-oversold-signals-explained/>
94. <https://www.markets.com/ph/education-centre/rsi-overbought-oversold-stocks/>
95. <https://www.quantifiedstrategies.com/rsi-trading-strategy/>
96. <https://www.schwab.com.hk/zh-hk/story/how-to-tell-if-market-is-overbought-or-oversold>
97. <https://www.tejwin.com/en/insight/rsi-indicator/>

98. <https://www.linkedin.com/pulse/combining-bollinger-bands-rsi-detect-overbought-oversold-conditions-8cekf>
99. <https://help.vestinda.com/article/35-simple-bollinger-bands-and-rsi-trading-strategy>
100. <https://forextester.com/blog/bollinger-bands-rsi-stochastic-scalping-strategy/>
101. https://www.reddit.com/r/Daytrading/comments/1pt0eas/backtested_rsi_bollinger_bands_strategy_across/
102. <https://tradesearcher.ai/strategies/1615-bollinger-band-with-rsi>
103. <https://tradesearcher.ai/strategies/2625-rsi-b-signal-h1-backtesting>
104. <https://quantbacktest.com>
105. <https://www.quantinsti.com/articles/backtesting-trading/>
106. <https://www.exegy.com/avoiding-slippage-equities-trading-with-backtesting/>
107. <https://www.luxalgo.com/blog/backtesting-limitations-slippage-and-liquidity-explained/>
108. https://tw.tradingview.com/scripts/relativestrengthindex/page-17/?script_type=strategies
109. https://www.youtube.com/watch?v=teSI4_vpZz0
110. <https://algogene.com/community/post/28>
111. <https://kernc.github.io/backtesting.py/>
112. <https://github.com/kernc/backtesting.py>
113. <https://www.backtrader.com>
114. <https://www.interactivebrokers.com/campus/ibkr-quant-news/backtesting-py-an-introductory-guide-to-backtesting-with-python/>
115. <https://github.com/mccaffers/backtesting-engine-cpp>
116. <https://cran.r-project.org/web/packages/portfolioBacktest/index.html>
117. <https://www.quantstart.com/articles/backtesting-systematic-trading-strategies-in-python-considerations-and-open-source-frameworks/>
118. <https://cran.r-project.org/web/packages/backtest/backtest.pdf>
119. <https://pipekit.io/blog/python-backtesting-frameworks-six-options-to-consider>
120. <https://github.com/mementum/backtrader>
121. <https://cran.rstudio.com/web/packages/portfolioBacktest/portfolioBacktest.pdf>
122. https://www.reddit.com/r/algotrading/comments/1fi83nx/python_librarybacktesting/
123. https://mccaffers.com/quantitative_engineering/building_a_backtesting_system/
124. <https://www.youtube.com/watch?v=SnXj2yKa9Wc>
125. https://www.reddit.com/r/algotrading/comments/1hp131u/making_a_backtesting_engine_resources/
126. <https://www.robeco.com/docm/docu-the-volatility-effect-2007.pdf>
127. <https://finance.lab.nycu.edu.tw/Students/98周立軒/Common Risk Factors in the Returns on Stocks.pptx>
128. <https://pages.stern.nyu.edu/~adamodar/pdfiles/papers/beta.pdf>
129. <https://www.sciencedirect.com/topics/economics-econometrics-and-finance/beta-risk>

130. <https://en.antares-is.de/blog/risk-map-risikomanagement-effektive-visualisierung-und-bewertung-von-unternehmensrisiken>
131. <https://jbem.vgtu.lt/index.php/JBEM/article/view/3782>
132. https://www.reddit.com/r/StockMarket/comments/gkn64o/why_understanding_alpha_beta_drawdown_is_a/
133. <https://www.vision.cpa/blog/Charting the Course: Using Data Visualization for Cyber Risk Management>
134. <https://www.adviser.best/articles/en/blog/investment-strategies/multi-factor-investing-momentum-value-quality-size>
135. <https://fastercapital.com/topics/how-to-account-for-other-factors-that-affect-risk-and-return,-such-as-size,-value,-momentum,-and-quality.html>
136. <https://www.wealthease.in/post/factor-investing-in-india>
137. <https://eaminvestors.com/wp-content/uploads/2024/08/Momentum-and-Quality.pdf>
138. <https://www.sciencedirect.com/science/article/abs/pii/S0304405X15001774>
139. <https://jacobslevycenter.wharton.upenn.edu/wp-content/uploads/2015/05/The-Common-Factor-in-Idiosyncratic-Volatility.pdf>
140. https://www.youtube.com/watch?v=p3g_5tUymQw
141. https://ta-lib.github.io/ta-lib-python/func_groups/volatility_indicators.html
142. <https://tessl.io/registry/tessl/pypi-ta-lib/0.6.0/files/docs/volatility-indicators.md>
143. <https://opendeep.wiki/TA-Lib/ta-lib-python/volatility-indicators>
144. <http://www.mfbz.cn/a/203115.html>
145. https://ta-lib.github.io/ta-lib-python/func_groups/statistic_functions.html
146. <https://tessl.io/registry/tessl/pypi-ta-lib/0.6.0/files/docs/statistical-functions.md>
147. https://github.com/TA-Lib/ta-lib-python/blob/master/docs/func_groups/statistic_functions.md
148. <https://ithelp.ithome.com.tw/articles/10241117?sc=pt>
149. <https://dev.to/lemon-markets/10-useful-python-libraries-packages-for-automated-trading-4fbm>
150. https://blog.csdn.net/weixin_47339916/article/details/147370938
151. <https://blog.quantinsti.com/risk-constrained-kelly-criterion/>
152. https://blog.csdn.net/qg_39065491/article/details/134705988
153. <https://ithelp.ithome.com.tw/m/articles/10241117>
154. https://ta-lib-python.transdocs.org/func_groups/volatility_indicators
155. https://www.reddit.com/r/algotrading/comments/1kdp3mp/my_first_almost_complete_algo/
156. <https://pages.stern.nyu.edu/~adamodar/pdfiles/papers/beta.pdf>
157. <https://www.sciencedirect.com/topics/economics-econometrics-and-finance/beta-risk>
158. <https://jbem.vgtu.lt/index.php/JBEM/article/view/3782>
159. <https://www.robeco.com/docm/docu-the-volatility-effect-2007.pdf>
160. <https://www.sciencedirect.com/science/article/abs/pii/S0304405X15001774>
161. <https://jacobslevycenter.wharton.upenn.edu/wp-content/uploads/2015/05/The-Common-Factor-in-Idiosyncratic-Volatility.pdf>

162. <https://finance.lab.nycu.edu.tw/Students/98周立軒/Common Risk Factors in the Returns on Stocks.pptx>